

Bases de Datos

Práctica 2 - Sesión 2.

- La práctica se debe entregar en **DeliverIt** donde se realizarán una serie de pruebas unitarias automáticas.
- Para que los archivos de código fuente compilen se debe respetar estrictamente los nombres de las clases y paquetes.
- Adicionalmente, el alumno puede y debe comprobar el resultado del código fuente utilizando su propio entorno local.

1. Descripción general

En esta segunda sesión se **piden 2 clases**:

InsertaUnaFilaImparte. Inserta una fila en la tabla `imparte`.

InsertaImparteDesdeCSV. Sirve para añadir datos a la tabla `imparte` desde un archivo csv.

Es necesario utilizar las siguientes clases e interfaces:

1. Una interfaz, `DataBaseTask`. Igual que en la **Sesión 1**.
2. Una clase derivada de `Exception`, `BDDException`. Igual que en la **Sesión 1**.
3. La clase `StringWriter`. Igual que en la **Sesión 1**.
4. Las clase de gestión de conexiones, `BBDDManager` resuelta de la sesión anterior.
5. Una clase `Main` para hacer pruebas locales.

Es necesario utilizar la clase `BBDDManager` desarrollada en la Sesión 1, pero para el corrector se va a utilizar una versión correcta, de manera que no se va a volver a evaluar. Adicionalmente los alumnos pueden aprovechar esta sesión para comprobar el comportamiento de su `BBDDManager`, en el ejercicio para cargar los datos del archivo CSV, si al insertar en *auto commit false* provocan un error de `SQLException` deberían observar que no se produce ningún cambio.

Es necesario haber creado la tabla `imparte`. Sin embargo, en el corrector se va a utilizar una base de datos donde la tabla se ha creado correctamente. Para esta sesión solamente se debe asumir que la tabla tiene 4 campos enteros y una fecha cuyos nombres de columna son respectivamente y en este orden: `profesor_id`, `curso_id`, `n_modulo`, `aula_id` y `fecha`. Para comprobar que se producen errores será necesario añadir alguna clave primaria.

Para realizar la extracción de datos para `InsertaUnaFilaImparte`, se recomienda usar los métodos de instancia `String.split` y `String.trim` para dividir la cadena y quitar espacios. Para extraer la información del archivo para `InsertaImparteDesdeCSV` se recomienda usar las clases `FileInputStream` y un `Scanner` sobre él para leer línea a línea.

Advertencias

1. Los nombres en SQL son *case-sensitive* porque el servidor de Deliverit es un Linux. Es especialmente importante poner los nombres de las tablas en minúsculas.
2. El código en Java debe escribirse en ISO-8859-1. Para evitar problemas lo que sencillo es evitar todo tipo de caracteres especiales (tildes, eñes) en todo el contenido de los fuentes (incluyendo comentarios).

2. Modelo de Datos

2.1. Clases e interfaces

InsertaUnaFilaImparte.

```
package cursos;

import java.sql.*;
import java.time.LocalDate;

public class InsertaUnaFilaImparte implements DataBaseTask {
```

```

/**
 * Inserta los datos proporcionados en una nueva fila de la tabla 'imparte'.
 *
 * ATENCION: para poner la fecha se recomienda usar:
 * - java.time.LocalDate.of
 *     Para crear una LocalDate con year, month, day
 * - java.sql.Date.valueOf
 *     Para transformarla en Date
 *
 * @param conn La conexcion ya abierta
 * @param data Los datos a insertar en el formato: "7, 3, 2, 4, 14/03/2025",
 *     es decir, los datos pueden llevar espacios antes o despues.
 *     Cada valor es respectivamente el valor de las columnas:
 *     profesor_id, curso_id, n_modulo, aula_id, fecha.
 *
 * @throws BBDDException, si se produce cualquier error durante
 *     el proceso de los datos del archivo se debe fijar
 *     'when' a "Insertando"
 * @throws SQLException, cuando se produzca la misma al ejecutar la
 *     insercion o si la ejecucion del comando no retorna 1.
 */
@Override
public void run(Connection conn, String data) throws BBDDException, SQLException { ... }
}

```

InsertaImparteDesdeCSV.

```

package cursos;

import java.io.*;
import java.sql.*;
import java.time.LocalDate;
import java.util.Scanner;

public class InsertaImparteDesdeCSV implements DataBaseTask {

    /**
     * Lee de un CSV con datos de 'imparte' y los inserta en la BD.
     *
     * Crea un objeto para ejecutar el comando de SQL INSERT y
     * lee linea a linea el archivo para insertarlas en la tabla.
     *
     * - El archivo CSV no tiene etiquetas de columnas, las columnas son:
     *     profesor_id, curso_id, n_modulo, aula_id, fecha.
     * - Las columnas se separan por comas.
     * - El formato de la fecha es: yyyy-mm-dd (anho-mes-dia).
     *
     * Si se produce cualquier error se debe interrumpir el proceso.
     *
     * @param conn La conexcion ya abierta
     * @param data El nombre (ruta) del archivo CSV
     * @throws BBDDException, si se produce cualquier error durante
     *     el proceso de los datos del archivo se debe fijar
     *     'when' a "Insertando"
     * @throws SQLException, cuando se produzca la misma al ejecutar la
     *     insercion o si la ejecucion del comando no retorna 1.
     */
    @Override
    public void run(Connection conn, String data) throws BBDDException, SQLException { ... }
}

```